図で理解する kickmix

可逆回路・位相キックバック・軌跡シミュレーション

Qualtran リポジトリの実装と資料に基づく解説書

2026 年 10 月

本書の読み方

本書は、Qualtran リポジトリに同梱された `kickmix/` のソースコードと `docs/kickmix/` の仕様をもとに、量子回路の基礎から使い方、シミュレータの内部構造まで説明する。`ref/docs/explanation.tex` の章立て、学習目標、注意欄、TikZ による図解を参考にした。対象読者は線形代数と Python の初歩を知る人である。最初は第 1〜4 章を順に読み、実際に使う際は第 5〜7 章を参照してほしい。

注意

kickmix は任意の量子回路を状態ベクトルで追跡するシミュレータではない。基底状態上の可逆計算、対角位相、条件付き命令と、制約を満たす測定ベースの逆計算に特化する。後述の HMR の前提を外すと一般の量子力学として正しい結果は得られない。

目次

第 1 章　kickmix の位置づけ 1

第 2 章　基底値と位相の基礎 2

第 3 章　HMR と測定ベースの逆計算 3

第 4 章　命令とファイル形式 5

第 5 章　Python で動かす 7

5.1　CircuitBuilder と補助量子ビット 7

第 6 章　シミュレーションの仕組みと検証 9

第 7 章　再現と資料案内 10

第 1 章

kickmix の位置づけ

この章で学ぶこと

何をシミュレートし、何を検証できるかを区別する。

量子アルゴリズムの中には、整数加算、比較、有限体演算などを可逆回路として実行する部分がある。kickmix はそのような部分を小さな命令列に表し、多数の入力や測定結果について、出力と位相を調べる。回路生成・解析と軌跡の実行は分かれた機能である。

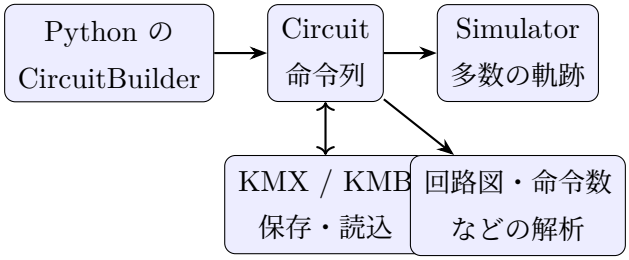


図 1.1　回路を作る、保存する、実行する、解析するという流れ。

n 量子ビットの一般の純粋状態は 2^n 個の複素振幅を持つ。一方、kickmix の一軌跡は各量子ビットの計算基底値、古典ビット、および位相を追う。回路が基底状態を別の基底状態へ写し、位相を付ける範囲では、この表現は各入力に対する変換の検査に適する。任意の Hadamard ゲートを含む汎用回路や、干渉のある任意の出力分布を計算する用途には使えない。

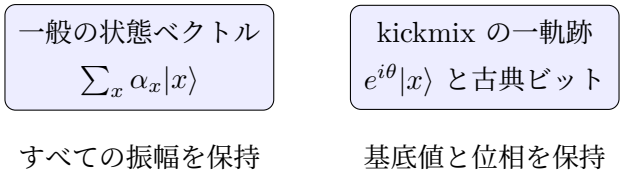


図 1.2　内部表現の違い。各軌跡は重ね合わせの全振幅を表すものではない。

第 2 章

基底値と位相の基礎

この章で学ぶこと

可逆ゲートと対角ゲートが一軌跡に及ぼす変化を追う。

計算基底を $|x\rangle$ とすると、可逆古典回路は $|x\rangle \mapsto |f(x)\rangle$ と書ける。f は置換である。対角ゲートは値を変えず位相を付ける。両者を組み合わせた回路は、測定のない範囲では

$$|x\rangle \longmapsto e^{i\theta(x)}|f(x)\rangle \tag{2.1}$$

の形で理解できる。入力によって位相が異なると、重ね合わせでは相対位相となり、後段で観測可能な差を生む。具体的には、CX : $(a,b) \mapsto (a, b\oplus a)$、CCX : $(a,b,t) \mapsto (a,b,t\oplus ab)$、

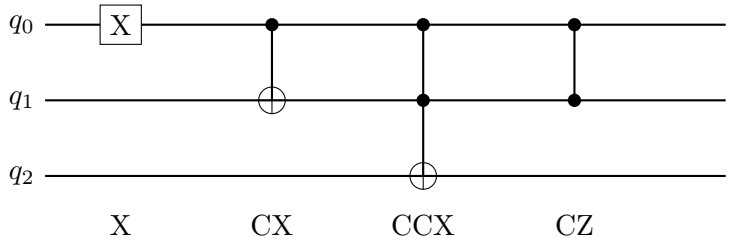


図 2.1　代表的な命令。黒丸は制御、丸に十字は反転対象。CZ は両方が 1 のとき位相を反転する。

CZ : $|a,b\rangle \mapsto (-1)^{ab}|a,b\rangle$ である。`Z_POWq0t` は $|1\rangle$ に $e^{i\pi t}$ を掛ける。一般の t に対して位相は ± 1 に限らない。フォーマットは指数を十進数で指定し、通常の倍精度浮動小数点に丸めるだけでは満たせない精度を要求する。

第 3 章

HMR と測定ベースの逆計算

この章で学ぶこと

一時量子ビットを消去する際に生じる位相と補正を導く。

可逆計算では一時値を残すと、入力と補助量子ビットが絡み合う。まず $t=0$ を用意して $t \leftarrow t \oplus ab$ とすると、重ね合わせ入力に対し

$$\sum_{a,b} \alpha_{ab} |a, b, 0\rangle \longmapsto \sum_{a,b} \alpha_{ab} |a, b, ab\rangle \tag{3.1}$$

となる。HMR は対象を X 基底で測定し、結果 $m \in \{0,1\}$ を古典ビットに保存して対象を $|0\rangle$ に戻す。$\langle m_X | t \rangle = (-1)^{mt}/\sqrt{2}$ より、測定後には $(-1)^{mab}$ が残る。$m=1$ のとき CZ を適用すると同じ符号をもう一度掛けるので消える。

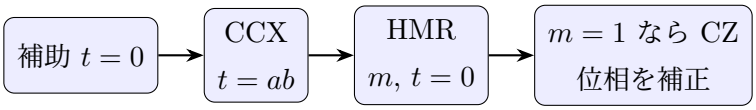


図 3.1　AND の計算と測定ベースの逆計算。測定値のフィードバックが必要。

```
R q2
CCX q0 q1 q2
HMR q2 b0
CZ q0 q1 if b0
```

注意

HMR は対象が他の量子ビットの計算基底値の関数である場合を想定する。この条件を満たさない場合、実装は無作為な結果と位相を使うが、一般の量子状態の正しい測定シミュレーションではない。誤用は位相を含むファズテストで確率的に検出する設計である。`R` を非ゼロの値に適用すると位相がランダム化されるため、通常はクリーンな補助量子ビットに使う。

`NEG` は全体符号を反転する。単独の量子状態の全体位相は観測できないが、kickmix は軌跡の位相検査を容易にするため、必要に応じてこの命令による補正も明示する。

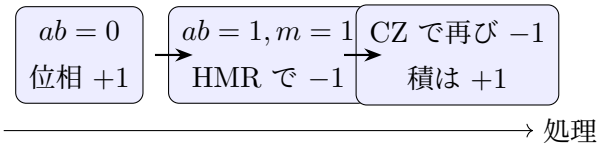


図 3.2　位相補正の例。$m = 0$ の場合は補正は不要。

第 4 章

命令とファイル形式

この章で学ぶこと

KMX の命令、古典制御、レジスタを読む。

KMX は UTF-8 のテキスト形式で、原則として 1 行に 1 命令を書く。`q` は量子ビット、`b` は古典ビット、`r` はレジスタの番号を表す。`ifb0` は量子制御ではなく古典ビットによる実行条件である。

分類	主な命令と役割
量子値の更新	`X`, `CX`, `CCX`, `SWAP`, `CSWAP`
位相	`Z`, `CZ`, `CCZ`, `Z_POW`, `NEG`
初期化・測定	`R`, `HMR`
古典値	`BIT_INVERT`, `BIT_STORE0`, `BIT_STORE1`
条件・メタデータ	`PUSH_CONDITION`, `POP_CONDITION`, `REGISTER`, `APPEND_TO_REGISTER`, `INFO_TAG`, `DEBUG_PRINT`

条件スタックは `PUSH_CONDITIONifb0` の時点の値を保存する。その後 `b0` を書き換えても保存済みの条件は変化しない。通常の命令はスタック上の全条件と、その命令自身の条件が真のときに実行される。条件スタック操作とレジスタ・情報タグのメタデータはスタックを無視する。レジスタへの追

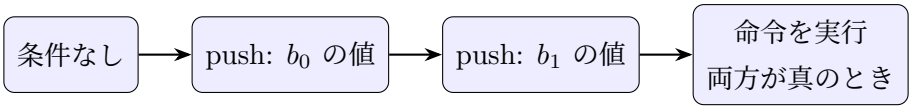


図 4.1　条件スタックの概念図。pop で直近の条件を取り除く。

加順は下位ビットからで、最初の要素が 2^0 の位を表す。レジスタは名前付きの参照・入出力のためのメタデータであり、宣言自体は量子状態を変えない。KMB は同じ回路をコンパクトなバイナリで

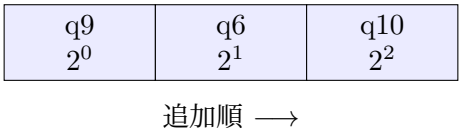


図 4.2　リトルエンディアンのレジスタ。量子ビット番号と桁の順序は別である。

保存する形式である。読み込み・書き出しは `Circuit.from_file` と `Circuit.to_file` を用いる。詳細な構文、指数の精度、バイナリの符号化は巻末の一次資料を参照する。

第 5 章

Python で動かす

この章で学ぶこと

回路を作り、シミュレータで結果を取得する。

Python 拡張をビルド・インストールした環境で次の例を実行する。`Circuit` に KMX テキストを渡し、1 ショットの初期値 $q_0 = q_9 = 0$ に対して X と CX を適用する。

```
import kickmix as km

circuit = km.Circuit("""
    X q0
    CX q0 q9
""")
sim = km.Simulator(batch_size=1)
sim.do(circuit)
print(sim.read_within_shot(km.q(0), shot_index=0)) # True
print(sim.read_within_shot(km.q(9), shot_index=0)) # True
```

この例では、$00 \to 10 \to 11$ と値が変わる。初期値を変えて検査するには `write_within_shot` を使う。各ショットの位相は `read_shot_phase` で確認できる。測定がある場合は測定結果ごとに補正後の値と位相を検査する。回路だけを用意した段階では何も実行されず、`sim.do(circuit)` で初めて状態が変わる。

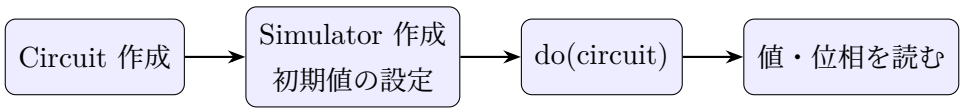


図 5.1　Python API の基本的な実行順序。

5.1　CircuitBuilder と補助量子ビット

`CircuitBuilder` は量子ビットの確保、命令の追加、加算や有限体演算などの回路生成を提供する。慣例上 `init_` はクリーンな対象への計算、対応する `del_` は逆計算を表す。`init_` は対象のリセットを含み、`"alloc"` による割り当てをサポートする。これらの前提と確保・解放の寿命を揃え

ることが重要である。配列を渡した基本ゲートは NumPy に似たブロードキャストを行う。メソッドごとの型と引数は API リファレンスを確認する。

第 6 章

シミュレーションの仕組みと検証

この章で学ぶこと

軌跡の並列処理とテスト結果の意味を理解する。

実装の `Sim<TWord,count_shots>` は、各量子ビット・古典ビットについて、複数ショットの値を一つのワードの各ビットに格納する。例えば CX は制御のビット列を対象のビット列に XOR すれば、複数ショットをまとめて更新できる。実際の幅は使用するワード型に依存する。位相には符号のビット列とショットごとの角度が使われる。入力の全組合せを試せる小規模回路では網羅的な確認が

q_0	1	0	1	1	0	0	1	0
q_1	0	1	1	0	1	0	0	1
条件	1	1	0	1	0	1	1	0

列ごとに独立したショット（模式図）

図 6.1　ビットスライス表現。行全体にビット演算を適用する。

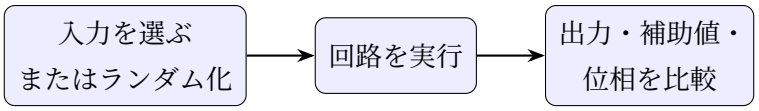


図 6.2　典型的な回路検証。出力値だけでなく不要な位相や補助値も確認する。

有効である。大規模回路のランダムテストは誤りの発見に役立つが、有限回の成功は正しさの証明ではない。特に HMR の補正漏れは測定結果と入力の組合せによって現れるため、多数の軌跡を使う意味がある。`operation_statistics` による命令数の分析と、実行された命令をショットごとに数える分析は別の観点である。

第 7 章

再現と資料案内

この章で学ぶこと

手元でビルドし、詳細仕様と実装をたどる。

リポジトリの開発資料では Bazel 7.2.0、pybind11、pytest を使用し、ルートから次の手順を示している。利用可能なコンパイラ・Python・Bazel 環境に応じて依存関係を整える。

```
bazel build kickmix:kickmix_dev_wheel
pip install bazel-bin/kickmix/kickmix-0.0.dev0-py3-none-any.whl
PYTHONPATH=kickmix pytest kickmix/src
bazel run kickmix:kickmix
```

CLI には `sample`、`diagram`、`count_operations`、`kmx2kmb`、`kmb2kmx` などがある。各サブコマンドのオプションと入出力はソースを確認する。`kickmix/testdata/` に KMX と KMB の例がある。

内容	リポジトリ内の資料
命令と意味	`docs/kickmix/kickmix_instructions.md`
テキスト・バイナリ仕様	`docs/kickmix/kickmix_format.md`, `docs/kickmix/kickmix_binary_format.md`
Python API と規約	`docs/kickmix/api_reference.md`, `docs/kickmix/conventions.md`
開発手順	`docs/kickmix/dev.md`
軌跡の実装	`kickmix/src/kickmix/sim/sim.h`, `sim.cc`
使用例と検証	`kickmix/src/kickmix/py/interactive_simulator_test.py` ほか
本書の体裁の参考	`ref/docs/explanation.tex`, `ref/docs/preamble.tex`